The Team's Approach vs Our Pipeline

M5 Retail Demand Forecasting — Problem Statement 11

What they planned, what we built, and what could explain the difference

NPN AIA Hackathon — St. Joseph's College of Engineering

Generated 2026-08-14. Read-only analysis: no model was trained, no pipeline code changed, no existing report overwritten.

The single most important thing to understand about the reference document. `end_to_end_approach.md` *is a* ***plan and a pitch****, not a record of what was built. It contains no train/test split, no validation dates, no forecast horizon, no hyperparameters, no Tweedie power, and no RMSE or MAE figures anywhere. The only metric it names is* ***WRMSSE****, not RMSE/MAE. So the numbers 2.0324 and 1.0869* ***cannot be traced to this document at all*** *— it does not tell us how they were produced.*

Plain-English glossary. RMSE *= average error, with big misses punished much more heavily.* ***MAE*** *= plain average error.* ***Leakage*** *= the model accidentally sees information from the future that would not have existed when the forecast was really made.* ***Lag*** *= a past value (lag_7 = sales seven days earlier).* ***Rolling mean*** *= average over a recent stretch of days.* ***Tweedie*** *= a loss function suited to data that is never negative and is mostly zeros.* ***SNAP*** *= a US food-assistance benefit; the calendar records which days it was usable in each state.*

Part 1 — What the team's document actually says

Feature / Method	Purpose	Team approach	Evidence in document	We use it?	Worth testing?
lag_7	weekly momentum	sales 7 days ago	Phase 1 feature table	Yes — but origin-relative	Already have
lag_28	monthly momentum	sales 28 days ago	Phase 1 feature table	Yes	Already have
rolling_mean_7	recent trend	7-day average	Phase 1 feature table	Yes — window ends at origin	Already have
rolling_mean_28	recent trend	28-day average	Phase 1 feature table	Yes — our strongest feature (74% of gain)	Already have
rolling_zero_count_7	intermittency	count of zero days in last 7	Phase 1 feature table	**No**	**Yes — cheap, plausible**
day_of_week	weekly rhythm	weekday index	Phase 1 feature table	Yes (`wday`)	Already have
month	seasonality	calendar month	Phase 1 feature table	Yes	Already have
day_of_month	payday effect	day number within month	Phase 1 feature table	**No**	**Yes — cheap, genuinely missing**
is_weekend	weekend surge	Sat/Sun flag; cites 31% surge	Phase 1 feature table	Yes	Already have
SNAP	benefit-day demand	`snap_CA/TX/WI` as three columns	Phase 1 feature table	Yes — but matched to each series' own state	Ours is stricter; no change
SNAP x FOOD interaction	food-specific SNAP lift	`is_food_and_is_snap`; cites 10.2%	Phase 1 feature table	**Not explicit** (model can learn it from snap + cat_id)	**Yes — cheap**
price_pct_change	price momentum	week-over-week % change	Phase 1 feature table	**No** (we use price ÷ own recent average)	**Yes — different construction**
phantom promotion	proxy for missing promo data	flag weeks where price fell >5%	Phase 1 feature table + Idea #3	**No**	Maybe — see Part 4

Feature / Method	Purpose	Team approach	Evidence in document	We use it?	Worth testing?
ghost stockout	exclude suspicious zeros	28-day avg >3/day AND today 0 AND prior 3 days 0; **exclude from training**	Phase 0 Step 2 + Phase 1	**No**	Test as a *feature*, not a deletion
leading-zero removal	drop pre-launch rows	mark days before first price as `pre_launch`; **remove entirely**	Phase 0 Step 1	We flag it, never delete	Test removal from training only
Christmas override	domain rule	force prediction to 0 on every Dec 25	Phase 0 Step 3	**No**	**No — see Part 4**
cat_id / dept_id / store_id	hierarchy	categoricals as embeddings	Phase 1 + Phase 2	Yes — plus `item_id` and `state_id`	Already broader
Global LightGBM	one model for all series	single global model	Phase 2	Yes	Already have
Tweedie	zero-inflated loss	Tweedie objective; power not stated	Phase 2	Yes (power 1.1; 1.3/1.5 also tested)	Already tested
Foods-first tuning	tune on the volume driver	tune hyperparameters on FOODS; others on defaults	Phase 2	**No**	Low priority — see Part 4
Bottom-up reconciliation	coherent hierarchy	sum store-item preds up 12 levels	Phase 2	**No**	**No — not required by the deliverable**

Claims in the document that do not match the data

These were checked directly against the raw files. They matter because three of them are the exact figures that were in dispute at the start of this project — this document is where they came from.

Document says	Actual, verified from raw files	Impact
"FOODS drives **69.56%** of total sales volume"	**68.62%**	Cosmetic. Does not change any modelling decision.
Melting creates "**~30 million** rows"	**59,181,090** rows	Cosmetic, but roughly 2x out — worth correcting in the pitch.
"training **42,840** separate models (one per series)"	**30,490** series. 42,840 is the sum of all 12 WRMSSE hierarchy levels	Cosmetic, but a judge who knows M5 may notice.
"The sales file has `d_1` to `d_1969` as column headers"	Sales files stop at `d_1941`; only `calendar.csv` reaches `d_1969`	**Not cosmetic.** Melting to `d_1969` would create 28 days of phantom rows with no sales. If those were filled with 0 and trained on, it would harm the model; if used as the prediction frame, it is harmless.
"**10.2%** SNAP shockwave"	Our EDA measured **+12.7%** overall and **+17.3%** within FOODS	The 10.2% figure closely matches an early CA-only spot check in `DATASET_SUMMARY.md`, so it looks like a state-specific number quoted as a global one.

Part 2 — What our pipeline actually does

Every value below is read from the repository, not from memory.

Setting	Value	Source
Forecast origin	d_1913 (2016-04-24)	`pipeline/config.py`
Validation days	d_1914 .. d_1941 (2016-04-25 .. 2016-05-22)	`pipeline/config.py`
Horizon	28 days	`pipeline/config.py`

Setting	Value	Source
Series	30,490	verified against raw files
Predictions scored	853,720	all series x all 28 days
Training window	15 origins, d_1493 .. d_1885 (420 contiguous days)	`experiments/model_04_*.json`
Training rows	12,805,800	same
Lags	[1, 7, 14, 28] — **origin-relative**	`pipeline/config.py`
Rolling windows	[7, 28] (mean and std), ending at the origin	`pipeline/config.py`
Recency	days_since_last_sale, zero_streak_length, days_since_first_sale	`pipeline/features.py`
Listing	days_since_first_listing, pre_listing	`pipeline/features.py`
Price	sell_price, recent_avg_price, price ÷ recent avg, price_is_missing	`pipeline/features.py`
Hierarchy	item_id, dept_id, cat_id, store_id, state_id (native categoricals)	`pipeline/features.py`
Objective	tweedie, variance_power = 1.1	`experiments/model_04_*.json`
Rounds / leaves / lr	400 / 128 / 0.05	`pipeline/models.py`
Seed	42, `deterministic=True`	`pipeline/models.py`
Clipping	predictions clipped at 0	`pipeline/models.py`
Strategy	direct multi-horizon, fixed origin (no recursion)	`pipeline/backtest.py`
Leakage test	future sales overwritten with 9999; all 32 features bit-identical	`pipeline/validation_checks.py`

Total features: 32, in 7 groups.

- **A_calendar** — `wday`, `month`, `year`, `is_weekend`, `event_name_1`, `event_type_1`, `event_name_2`, `event_type_2`, `snap`
- **B_historical_demand** — `lag_1`, `lag_7`, `lag_14`, `lag_28`, `rolling_mean_7`, `rolling_mean_28`, `rolling_std_7`, `rolling_std_28`
- **C_recency** — `days_since_last_sale`, `zero_streak_length`, `days_since_first_sale`
- **D_listing** — `days_since_first_listing`, `pre_listing`
- **E_price** — `sell_price`, `recent_avg_price`, `price_rel_to_recent_avg`, `price_is_missing`
- **F_hierarchy** — `item_id`, `dept_id`, `cat_id`, `store_id`, `state_id`
- **G_horizon** — `horizon`

Part 3 — Side by side

#	Topic	Team document	Our pipeline	Difference	Should test?
1	Feature definitions	listed by name only, no formulas	explicit formulas, unit-checked	theirs under-specified	n/a
2	Lag definitions	lag_7, lag_28 in a melted long table	lag_1/7/14/28 **frozen at the origin**	**the deepest difference — see Part 5**	**already measured**
3	Rolling windows	rolling_mean_7/28 in a melted table	windows **end at the origin**, held constant	same as above	measured
4	Origin-relative?	**not mentioned anywhere**	yes, enforced and tested	unknown vs verified	n/a
5	Price features	price_pct_change (week over week)	price ÷ own 8-week average, plus missing flag	different construction	**Yes — cheap**

#	Topic	Team document	Our pipeline	Difference	Should test?
6	SNAP	three raw columns snap_CA/TX/WI	one flag matched to each series' own state	ours is stricter	No — ours is better
7	Calendar	day_of_week, month, day_of_month, is_weekend	wday, month, year, is_weekend, 4 event fields	**they have day_of_month, we do not**	**Yes — cheap**
8	Intermittency	rolling_zero_count_7	days_since_last_sale, zero_streak_length	different encoding of the same idea	**Yes — cheap**
9	Ghost stockouts	**delete those rows from training**	never delete anything	philosophical	Test as a feature only
10	Leading zeros	**delete pre-launch rows**	flag, never delete	philosophical	Test removal from training
11	Christmas	hard override to 0 on Dec 25	no override	**irrelevant here — neither window contains Dec 25**	**No**
12	Categoricals	cat_id, dept_id, store_id	those plus item_id and state_id	ours is broader	No
13	Objective	Tweedie, power unstated	Tweedie, power 1.1 (1.3 and 1.5 also measured)	we tested more	Done
14	Hyperparameters	**not stated**	fully recorded	unknown vs known	n/a
15	Training sample	everything except deleted rows	15 origins x 28 days = 420 contiguous days	different sampling	Maybe — more history
16	Validation	**not stated anywhere**	fixed origin d_1913, 28 days, 30,490 series	**unknown vs fully specified**	**must ask them**
17	Forecast strategy	not stated	direct multi-horizon	unknown	n/a
18	Clipping	not stated	clip at 0	unknown	n/a
19	Reconciliation	bottom-up across 12 levels	none	not required by submission format	**No**
20	Foods-first tuning	tune on FOODS, defaults elsewhere	one global setting	different tuning target	Low priority

Part 4 — Which of their ideas actually hold up

Ratings: **A** = strongly supported, safe to test · **B** = reasonable, needs testing · **C** = risky/unverified · **D** = do not use without much better evidence.

Global LightGBM — A

Correct, and we already do it. One model across 30,490 series lets sparse items borrow patterns from thousands of others. Their reasoning is sound (though the count is 30,490, not 42,840).

Tweedie — A

Correct and independently confirmed by us: switching only the objective improved our RMSE from 2.1467 to 2.1256. Their sentence "no other loss function handles this correctly" is too strong — Poisson and plain regression both work, just less well — but the choice is right.

SNAP x FOOD interaction — B

The effect is real: our EDA measured +12.7% overall and +17.3% within FOODS, and it lands exactly where domain knowledge predicts. A tree model can already discover this from `snap` and `cat_id` together, so an explicit product term may add little — but it is one line of code and worth a test.

Leading-zero (pre-launch) removal — B for training, D for evaluation

The underlying fact is real and we confirmed it *more strongly* than they did: rows before an item's first recorded price have a **100.00%** zero-sales rate. Removing them from **training** is defensible. But two things must be said plainly. First, at our forecast origin **0% of rows are pre-launch**, so this cannot change the forecast — by 2016 every item has long since launched. Second, removing them from **evaluation** would silently change the denominator and make scores incomparable.

Ghost stockout detection — C

The rule (28-day average >3/day, today 0, previous 3 days 0) is a reasonable heuristic, but the document calls the result a stockout. **The dataset has no inventory field, so a stockout cannot be confirmed.** A genuinely dead item, a delisting, or a bad demand week all produce the same pattern.

We applied their exact rule to our validation window: it flags **693 rows (0.0812%)**, on which our model predicts 6.038 units against an actual of zero. Those rows carry **0.888%** of our total squared error.

> ***This matters for the comparison.*** *Deleting rows from *training* is a modelling choice. Deleting them from *evaluation* is not — it removes precisely the rows a model is worst on. We measured that: excluding them moves our RMSE from **2.1210** to **2.1125**, about 10% of the gap to their figure. Real, but not the main story — and it moves MAE *down* to 1.0279, away from their higher MAE.*

Phantom promotion — C

A price drop >5% is a *price drop*, not a confirmed promotion. Our own EDA found sales rose after both price increases (+71%) and decreases (+48%) with a median effect of zero, meaning price changes often coincide with something else rather than causing it. Usable as a weak signal; must never be described to judges as promotion detection.

Christmas override — D (for this task)

The finding is real — Christmas is −99.95% versus a local baseline, stores are shut. But **neither our validation window (2016-04-25 to 2016-05-22) nor the actual forecast window (2016-05-23 to 2016-06-19) contains a December 25.** The override cannot change a single prediction. It is a good slide and a zero-impact feature; spending time on it would be time not spent on the 61% of error sitting in high-volume series.

Foods-first tuning — C

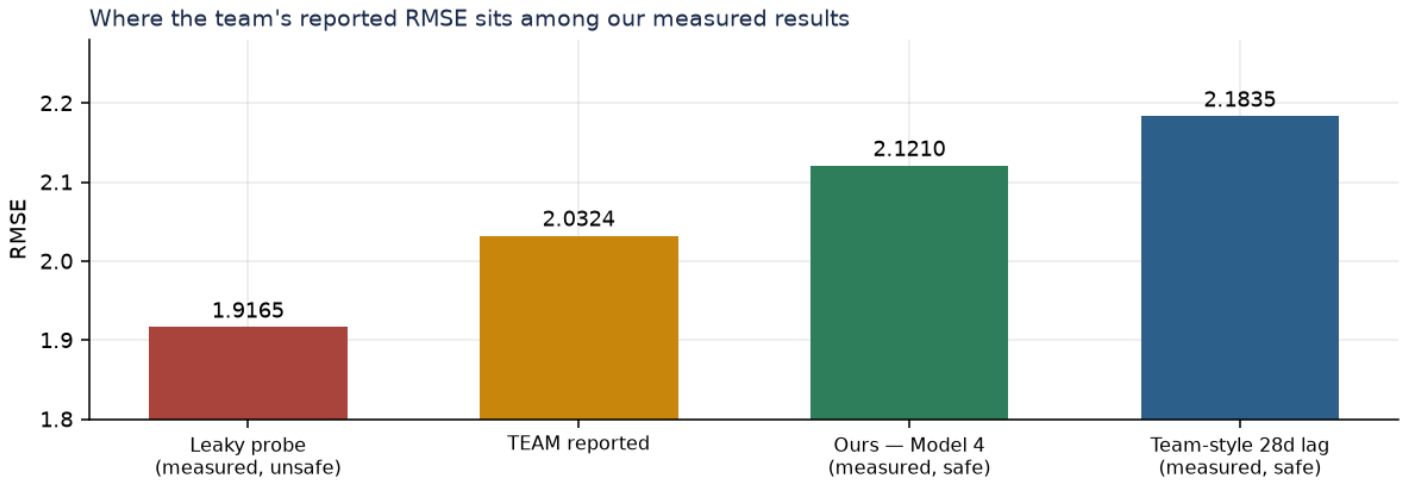
Justified in the document by WRMSSE being volume-weighted. But we are being compared on plain RMSE and MAE, which are not volume-weighted, so the premise does not transfer. FOODS does dominate our error (74% of it), so weighting *might* help — but that is a different argument than the one the document makes.

Bottom-up hierarchical reconciliation — D

`sample_submission.csv` asks only for store-item forecasts. Summing them upward is arithmetic that changes **none** of the 853,720 numbers being scored. It cannot improve RMSE or MAE by even a rounding error. It is presentation value only.

Part 5 — What could explain their 2.0324

Ranked by how much evidence supports each, highest first. **Nothing here is proven.** The document is silent on validation, so every explanation is a hypothesis about a method we have never seen.

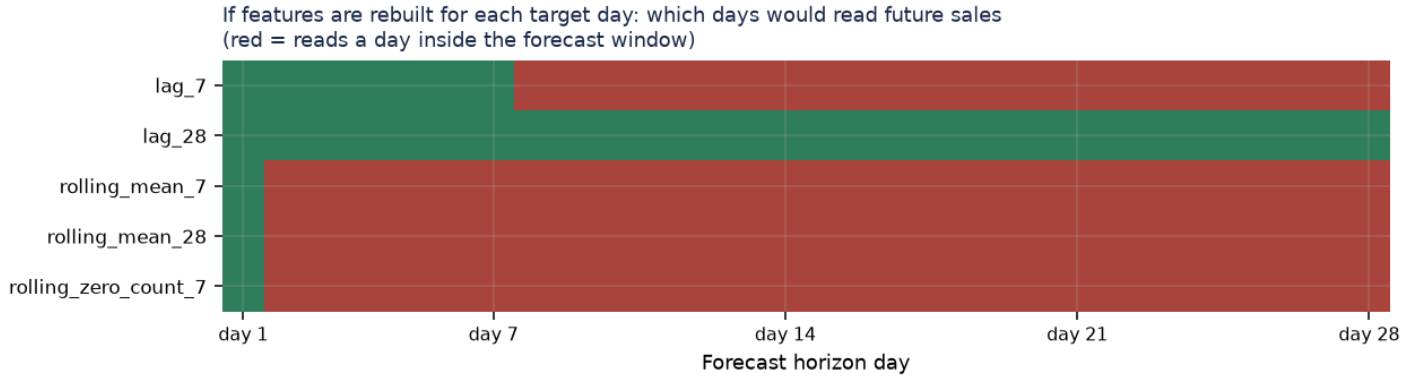


1. Features rebuilt per target day, without freezing at the origin — most likely

This is the one hypothesis the document actively supports. It says: melt to long format, one row per (item, store, day), then build `lag_7`, `rolling_mean_7`, `rolling_mean_28`, `rolling_zero_count_7`. It never mentions freezing those values at a forecast origin.

If features are computed per row in a melted table and the model then predicts 28 days at once, most of the horizon reads sales that had not happened yet:

Feature	Reads	Leaks on
`lag_7`	day t–7	days 21 of 28 (from day 8 onward)
`rolling_mean_7`	days t–7 … t–1	days 27 of 28 (from day 2 onward)
`rolling_mean_28`	days t–28 … t–1	days 27 of 28
`rolling_zero_count_7`	days t–7 … t–1	days 27 of 28
`lag_28`	day t–28	**0 of 28 — safe**



We already measured what this is worth. Our deliberately-leaky diagnostic probe scored **RMSE 1.9165** where our safe model scores **2.1210**. Their **2.0324** sits between the two — and that is exactly where a *milder* leak would land, because their shortest lag is `lag_7` rather than the `lag_1` our probe used.

> ***Stated carefully: this is a hypothesis, not an accusation.*** *The document does not say how they validated. It is entirely possible they froze features correctly and simply did not write it down. What we can say is that the mechanism is consistent with everything the document does describe, and it is the only explanation we have that produces a number in their range. It is a reason to ask them one specific question, not a verdict.*

2. A different evaluation population — plausible, partly measured

The document instructs that pre-launch rows and ghost-stockout rows be removed. If that removal also touched the evaluation set, the two scores are not measuring the same rows. We measured several variants on our own predictions:

Rows scored	n	RMSE	MAE
all rows (our reported figure)	853,720	2.1210	1.0319
exclude ghost-stockout rows	853,027	2.1125	1.0279
exclude every zero-actual row	388,995	2.9348	1.5660
exclude series with no sales in the window	830,984	2.1491	1.0551
FOODS only	402,360	2.6615	1.3369

Ghost-stockout exclusion moves RMSE in the right direction but explains only a fraction of the gap, and pushes MAE the wrong way.

3. A different validation window — largely ruled out

Their MAE is *higher* than ours, and MAE scales with how busy the period is. We measured the mean daily sales of every 28-day window in the last two years: the range is 1.0622 to **1.4428**, and the highest is our own window. There is no busier window for them to have used.

4. Genuinely different features — possible but small

They have three features we lack: `day_of_month`, `rolling_zero_count_7`, and an explicit SNAP×FOODS term. Our own ablation showed that everything beyond recent-demand features moved RMSE by hundredths, so a realistic expectation here is 0.00–0.02, not 0.09.

5. Hyperparameters — possible, unknown

Theirs are not stated. Ours are untuned by design. Our capacity search found more rounds made things *worse*, so this is unlikely to be worth 0.09.

6. A different metric implementation — cannot be excluded

The document names **WRMSSE** as the metric, not RMSE/MAE. If the reported figures were computed per-series and averaged, or on aggregated totals, or on a subset, they are simply a different quantity from ours.

Ruled out by measurement

- **Prediction clipping or calibration.** No rescaling of our predictions reaches below 2.1195; scaling up worsens both metrics.
- **Per-target-day features done *safely*** (28-day minimum lookback). We built and measured it: **2.1835**, worse than ours.
- **Bottom-up reconciliation.** Mathematically cannot alter store-item predictions.
- **Christmas override.** Neither window contains December 25.

> ***The part still unexplained by any hypothesis.*** *Their MAE (1.0869) is worse than every configuration we have measured — worse than ours (1.0319), worse than the safe team-style build (1.0498), and worse than the leaky probe (0.9754). Leakage improves both metrics, so leakage alone does not explain a *worse* MAE. The most likely reading is that two things differ at once: something that lowers their RMSE, and a base model or evaluation population that raises their MAE.*

Part 6 — The experiment ladder, and what to skip

All of these would use our exact validation window, unchanged target, and unchanged leakage controls, changing one factor at a time.

Exp	Change	Cost	Expected value	Verdict
A	Current best (reference)	none	—	**Already done** — RMSE 2.1210
B	Ask the team 5 questions about their validation	minutes	**decisive**	**DO THIS FIRST**
C	+ `day_of_month`, `rolling_zero_count_7`, SNAP×FOODS	~5 min	small but real	**RUN**
D	+ `price_pct_change` / phantom-promo flag	~5 min	small	**RUN** (bundle with C)
E	Exclude pre-launch rows from **training**	~5 min	small	**RUN** — settles a live question
F	Ghost-stockout flag as a **feature** (never deleted)	~5 min	small	**RUN** (bundle with E)
G	Recursive forecasting (feed own predictions back as lags)	~1–2 h	**largest legitimate upside**	**RUN if time**
H	Christmas override	~5 min	**exactly zero**	**SKIP** — no Dec 25 in either window
I	Bottom-up reconciliation	~30 min	**exactly zero on the metric**	**SKIP** for accuracy
J	Foods-first tuning	~1 h	unclear	**SKIP for now**
K	Team categorical set (drop item_id/state_id)	~5 min	likely negative	**SKIP** — ours is a superset

Bundle C+D and E+F into two runs rather than four. Our ablation showed individual feature groups move RMSE by hundredths, so testing them one at a time costs more time than the information is worth.

Part 7 — Recommended path

Keep from our pipeline

- The **fixed-origin design**. We tested the alternative and it was worse (2.1835 vs 2.1210).
- The **empirical leakage test**. It is the single most defensible thing in this project, and it already caught one real issue.
- **Tweedie**, global LightGBM, our broader categorical set, our state-matched SNAP flag, and clipping at zero.
- **Never deleting rows** from evaluation.

Borrow from the team

- `day_of_month` — genuinely missing from ours, and payday cycles are real.
- `rolling_zero_count_7` — a different, possibly better encoding of intermittency.
- Explicit SNAP×FOODS term — cheap to add.
- `price_pct_change` — a different price construction from ours.
- Pre-launch removal **from training only** — worth one clean test.

Test first

- **Ask them the five questions** (Part 6, Exp B). Nothing we build competes with simply learning their validation setup.
- **Bundle C+D** — four cheap features in one run.
- **Bundle E+F** — training-set filtering.
- **Recursive forecasting** if time remains.

Absolutely avoid

- **Copying their per-target-day lag construction to chase the score.** If our leading hypothesis is right, that number is only reachable by reading the future. A forecast that needs tomorrow's sales to predict tomorrow is worthless in production, and a judge who asks one careful question will expose it.
- **Deleting rows from the evaluation set.**
- **Christmas override and reconciliation** as accuracy work — both are provably zero-impact here.
- **Claiming ghost stockouts are detected.** No inventory field exists.

Highest-probability path to a genuinely better RMSE

Not feature dumping. Our error analysis is unambiguous: **high-volume series are 7.7% of rows and carry 61% of all squared error**, and we under-predict them (bias –0.389). The ranked options are:

- **Recursive forecasting** — the leaky probe puts an upper bound of about 0.20 RMSE on the value of fresher in-horizon information. Recursion captures part of that legitimately.
- **Volume-aware training** — weighting or a dedicated high-volume model.
- **The cheap features above** — worth having, but expect hundredths.

Most likely to waste time

Christmas override, hierarchical reconciliation, phantom-promotion engineering, Foods-first tuning, and re-testing recency or listing features that we have already measured twice as no help.

The four questions, answered directly

What did the team do? We know what they *planned*: melt to long format, build lag/rolling/calendar/price/SNAP features, delete pre-launch and suspected-stockout rows, train one global LightGBM with Tweedie loss, tune toward FOODS, reconcile bottom-up, and wrap it in an API, dashboard and GenAI copilot. We do **not** know how they validated it, and the document contains no RMSE or MAE at all.

What are we doing differently? Chiefly one thing: we freeze every history-derived feature at the forecast origin and prove by experiment that no feature moves when the future is altered. We also never delete rows, we match SNAP to each series' own state, and we carry two extra categoricals.

What should we test next? Ask them five questions; then one run adding `day_of_month` + `rolling_zero_count_7` + SNAP×FOODS + `price_pct_change`; then one run on training-set filtering; then recursive forecasting.

What could realistically close the 0.0886 RMSE gap? On the evidence, possibly nothing — because the gap may not be real. Our best current explanation is that their features read into the forecast window, in which case the number is not reproducible by any valid method. Of the legitimate levers, recursion is the only one plausibly worth that much; the new features are worth hundredths, not tenths.

What should we not copy? Per-target-day lags without origin freezing, row deletion from evaluation, the Christmas override, bottom-up reconciliation, and any language claiming stockouts or promotions have been detected.

Read-only analysis. No model trained, no pipeline modified, no existing report overwritten. Measured figures come from `experiments/` and from recomputation over `predictions/model_04_tweedie_recency_listing_validation.csv`.